AI HARNESS · 5H LECTURE

AI Harness

AI에게 일을 맡기기 전에 만들어야 하는 것

30년째 코드를 짜고 있는 개발자가, AI와 일하는 방식을 다시 설계한 기록

진짜 질문은 ‘쓰느냐’가 아니다

내 코드베이스의 어디까지를 AI에게 넘길 것인가?



Andrej Karpathy
@karpathy

There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper so I barely even touch the keyboard. I ask for the dumbest things like "decrease the padding on the sidebar by half" because I'm too lazy to find it. I "Accept All" always, I don't read the diffs anymore. When I get error messages I just copy paste them in with no comment, usually that fixes it. The code grows beyond my usual comprehension, I'd have to really read through it for a while. Sometimes the LLMs can't fix a bug so I just work around it or ask for random changes until it goes away. It's not too bad for throwaway weekend projects, but still quite amusing. I'm building a project or webapp, but it's not really coding - I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works.

8:17 AM · Feb 3, 2025 · **7.3M** Views

카파시는 가능성을 선언했다

- 자연어로 말하면 코드가 나오는 시대
- 코드의 존재를 잊고 흐름에 몸을 맡긴다

$ git show --format=fuller 93a7256

토발즈의 커밋 원문

```
Merge branch 'antigravity'

This is Google Antigravity fixing up my visualization tool (which was
also generated with help from google, but of the normal kind).

It mostly went smoothly, although I had to figure out what the problem
with using the builtin rectangle select was. After telling antigravity
to just do a custom RectangleSelector, things went much better.

Is this much better than I could do by hand? Sure is.
```

사실과 해석을 분리해 읽는다

커밋이 말하는 사실

- visualize.py 한 파일만 바뀌었다
- RectangleSelector를 커스텀으로 교체했다
- 본인이 평가한 뒤 직접 머지했다

여기서부터는 내 해석

- 전문 밖 주변부를 위임했다
- 막힐 때 답이 아니라 방향을 줬다
- 품질 기준은 여전히 사람이 쥐고 있다

한 커밋 안에 harness가 있었다

(되돌린 수 있는)경계 → **(손될 범위)격리** → **개입** → **평가 기준**

브랜치를 따로 파서
되돌릴 수 있게 했다

visualize.py 한 파일로
손댈 범위를 묶었다

막힐 때 커스텀 구현을
지시했다

손으로 짠 것보다
나은가로 판정했다

이 네 가지를 묶어 **harness**라고 부른다

커밋에는 사람의 이름이 남는다

책임은 도구가 아니라 머지한 사람에게 있다.

규칙에서 확률로

유창함과 정확함은 다른 속성이다

OPTIONAL

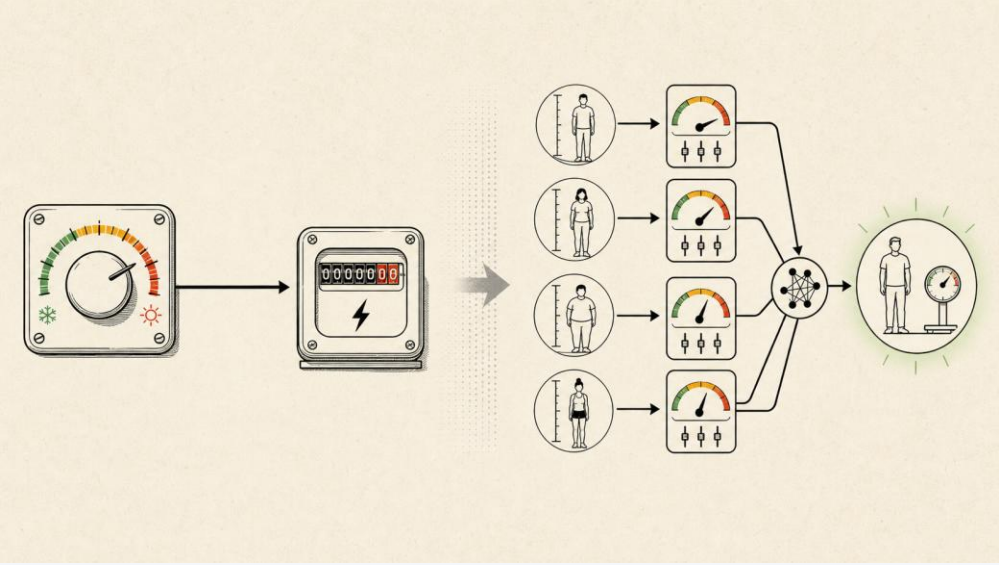
규칙은 사람이 쓰고, 학습은 관계를 조정한다

규칙 기반

- 조건을 사람이 정한다
- 틀리면 코드를 찾아 고친다
- 동작 이유를 설명하기 쉽다

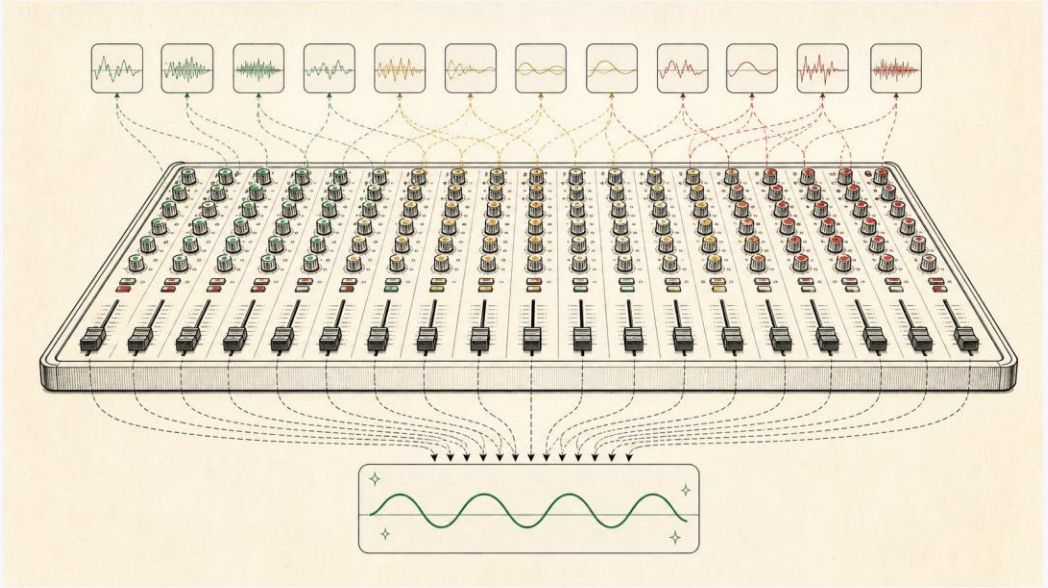
학습 기반

- 데이터로 오차를 줄인다
- 관계를 수치로 조정한다
- 데이터의 한계를 물려받는다



파라미터는 지식 목록이 아니다

- 하나의 값이 하나의 규칙을 뜻하지 않는다
- 수많은 값의 조합이 출력을 만든다

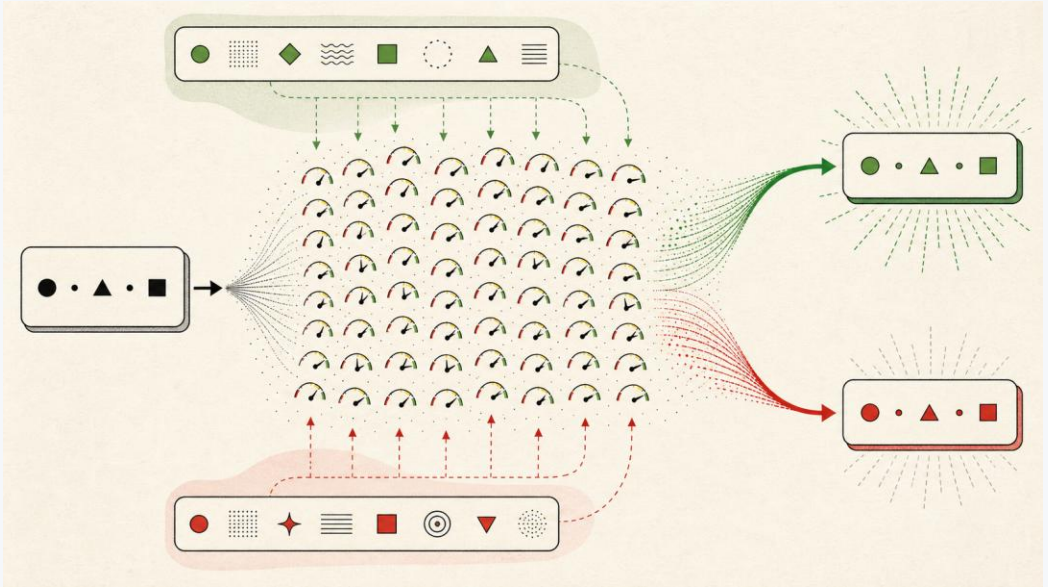


OPTIONAL

같은 입력도 맥락에 따라 후보가 달라진다

“휴스턴은…”

- 텍사스에 있다
- NASA가 있다
- 아주 덥다



OPTIONAL

유창함과 정확함은 다른 속성이다

- 그럴듯한 문장 ≠ 이해의 증거
- 자연스러운 번역 ≠ 사실 확인

자연어만 번역 결과가 보장되지 않는다

컴파일러

- 같은 입력
- 같은 번역 결과
- 재현 가능한 계약

자연어 → LLM

- 같은 문장
- 다른 코드가 가능
- 계약을 따로 만들어야 함

OPTIONAL

실무에서는 네 방식으로 흔들린다

- 할루시네이션 — 없는 API를 만든다
- 과잉 구현 — 요청하지 않은 것까지 만든다
- 누락 — 필요한 조건을 빼먹는다
- 왜곡 — 요구사항의 뜻을 바꾼다

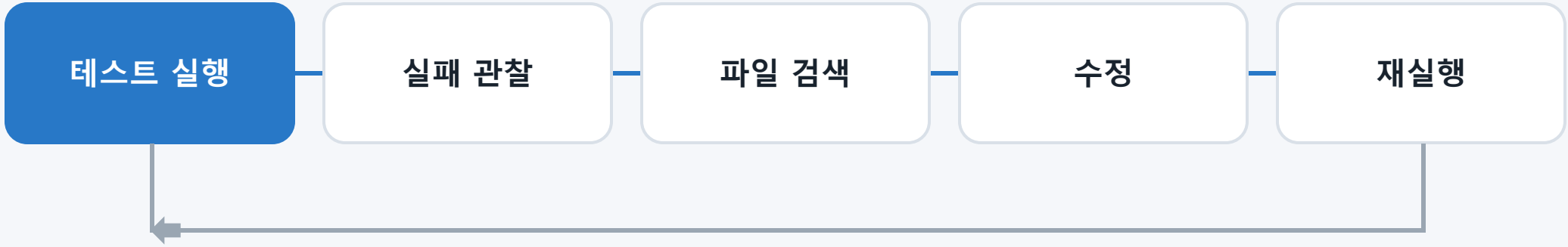
AI를 더 좋은 컴파일러로 취급하면 실패한다

- 결과가 흔들리는 도구는 믿음으로 다루지 않는다
- 행동하게 하고, 관찰하고, 검증한다

```
generate → observe → verify
```

관찰과 검증을 루프로 돌리면 Agent가 된다

예 — 테스트 실패를 고치는 루프



LLM은 텍스트를 생성한다

Agent는 다음 행동을 결정하며 루프를 돈다

판단 지점을 사람이 설계한다

01 Agent는 판단을 흉내 낸다

02 사람은 위임 경계와 실패 비용을 저울질한다

03 검증 가능한 만큼만 맡긴다

판단력은 모델이 아니라 환경에서 나온다

환경이란 AI가 건드릴 범위, 지켜야 할 규칙, 통과해야 할 검증이다

Loop와 Graph는 harness 위에 세워진다

그 위에서 도는 방법론



베이스 — 무엇을 어떻게 줄 것인가

루프가 잘못 돌 때 손댈 수 있는 층이 harness다
그래서 오늘은 harness engineering에 집중한다

목표는 인간 없는 루프가 아니다

AI가 맡는 일

- 코드 생성
- 도구 실행
- 검증 결과 관찰
- 수정과 재검증

사람이 맡는 일

- 목적과 범위
- 위임 경계
- 승인과 우선순위
- 최종 책임

Harness는 AI Agent가 일하는 환경이다

- harness는 원래 마구(馬具)다 — 말을 느리게 하는 장치가 아니라, 힘을 수레에 전달하는 장치다

- AI도 마찬가지다. 제약하는 게 아니라, 출력을 쓸모 있는 방향으로 전달한다

다섯 축은 다섯 개의 질문이다

권한	도구	**검증**	상태	관측
무엇을 건드릴 수 있는가	무엇을 쓸 수 있는가	맞다고 어떻게 확신하는가	무엇을 기억하는가	무엇을 했는지 어떻게 아는가
파일 범위 · 브랜치 격리 · 승인 모드	빌드 · 테스트 · 린터	테스트 · 타입 체크 · 리뷰 게이트	컨텍스트 · 문서 · 메모리	로그 · diff · 트레이스

harness는 도구의 목록이 아니라, 이 질문들에 답을 갖고 있느냐다

OPTIONAL

도구를 새로 만들지 않아도 harness는 작동한다

- 권한 — 브랜치 격리
- 검증 — 손으로 짠 것보다 나은가
- 관측 — 시행착오를 커밋 메시지에 기록

다섯 축 중 하나만 고른다면 검증이다

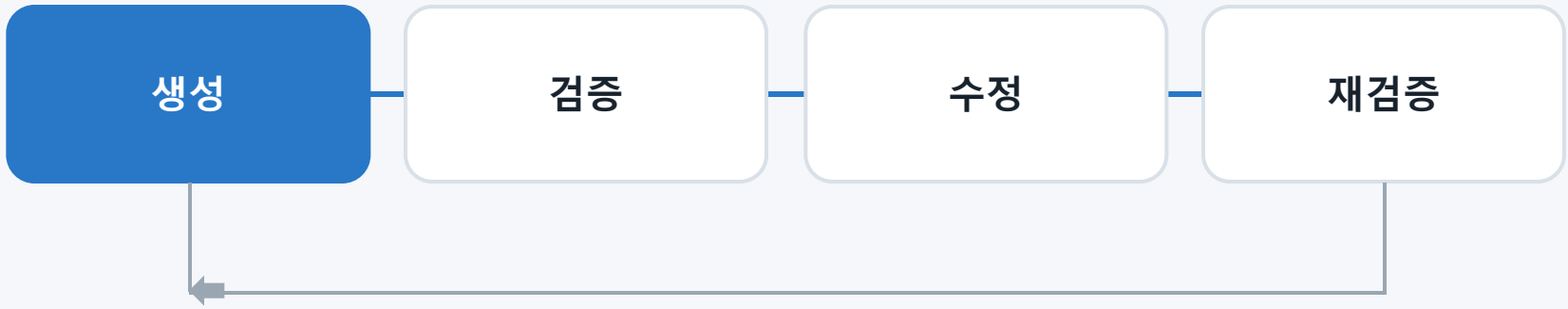
AI가 만든 코드를 신뢰하는 방법은 믿지 않는 것이다

대신 검증한다.

검증은 테스트 · 타입 체크 · 리뷰 게이트로 한다

검증 결과가 자동으로 되먹임되어야 한다

몇 번 돌릴지·언제 멈출지는 loop engineering이다. 여기서는 돌 수 있는 파이프가 있느냐만 본다.



이 경로가 자동이면 — harness가 도는 루프
사람이 복사해 붙여넣으면 — 그 파이프를 사람이 대신한다

실패가 다음 행동의 입력이 된다

OPTIONAL

승인과 전달은 다른 일이다

사람이 해야 할 승인

- 이 diff를 머지할 것인가
- 이 위험을 받아들일 것인가
- 요구사항에 맞는가

자동화해야 할 파이프

- 테스트 결과 전달
- 린터 오류 전달
- 실패 로그 전달

복붙은 판단이 아니라, 없는 파이프를 사람이 대신하는 것이다

누가·무엇을·어떻게를 분리한다

다섯 축은 환경이 갖출 속성이고, 이건 그 환경을 파일로 쪼개는 기준이다

미리 설계한다

- Agent — 누가
- Skill — 어떻게

매번 달라진다

- Task — 무엇을
- 오늘은 마이그레이션
- 내일은 버그 수정

뭉치면 재사용이 안 되고, 실패했을 때 어디가 문제인지 좁힐 수 없다

OPTIONAL

판정 문장을 쓸 수 있으면 게이트가 된다

harness가 무엇인지 알았다면, 남은 질문은 하나다 — 그 안에서 무엇을 강제할 것인가
여기서는 코드 수준. 프로세스 수준(SDD · DDD · TDD)은 다음 장이다

자동 게이트

명령어가 통과/실패를 판정한다

- DRY — 중복률 임계치 초과 시 실패
- SRP — 파일 400줄 초과 시 실패
- SoC — 레이어 역참조 시 실패

규칙으로 유도

판정은 되지만 우회할 수 있다

- YAGNI — 변경 파일 수 상한
- KISS — 복잡도 임계치
- Rule of Three — 성급한 추상화 금지

사람이 판단

판정 문장을 쓸 수 없다

- Meaningful Naming — 이 도메인에서 맞는 이름인가

칸은 고정이 아니다. 판정 문장을 쓸 수 있게 되는 순간 왼쪽으로 옮겨간다

원칙만으로 해결되는 실패는 없다

- 과잉 구현 → YAGNI가 줄인다
- 누락 → 검증 게이트가 잡는다
- 왜곡 → 용어집이 줄인다
- 할루시네이션 → 실행해 봐야 안다

루프에는 종료 조건이 필요하다

- 토큰은 무한하지 않다
- 정확한 경계와 함께 쓴다
- 실패 횟수·시간·승인 지점을 명시한다

책임질 수 있게 만드는 장치가 harness다

01 환경은 다섯 축으로 본다

02 핵심은 검증의 자동 되먹임이다

03 판단과 승인은 사람에게 남는다

여기까지가 1편이다. 남은 셋은 이렇게 이어진다

harness가 무엇인지 알았다면, 남은 질문은 하나다 — 그 안에서 무엇을 강제할 것인가

01 무엇을 강제할 것인가 — SDD · DDD · TDD · Deep Module

02 어떤 흐름으로 돌 것인가 — 합의 → 계획 → 구현 → 복원 → 지식

03 어떤 파일로 만들 것인가 — Agent와 Skill

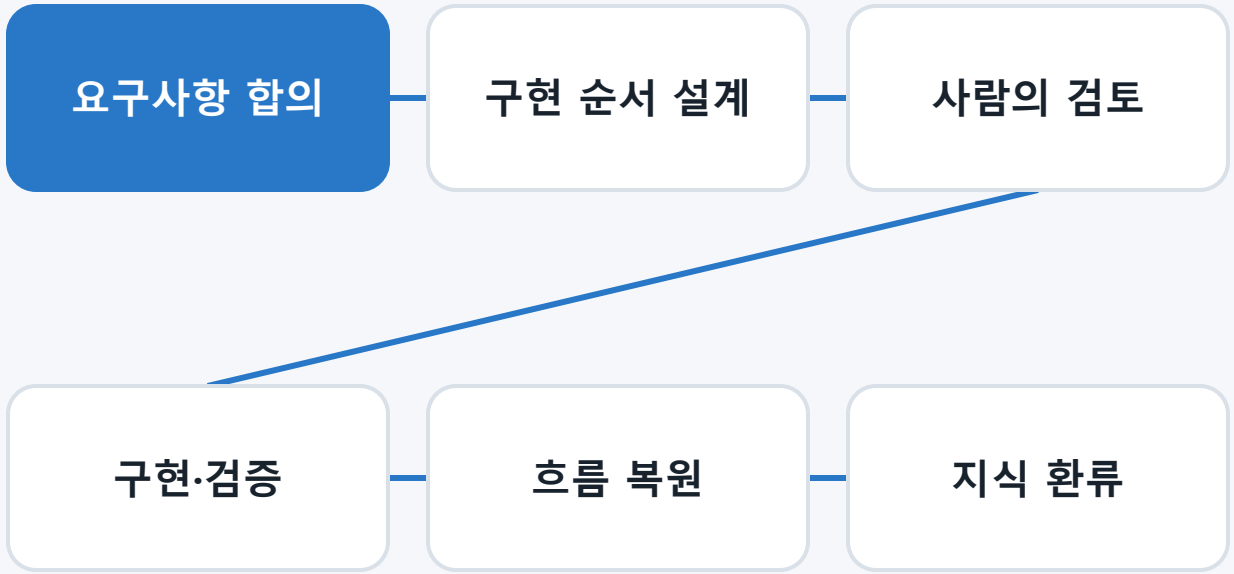
도구부터 배우면, 왜 그 단계가 거기 있는지 모른 채 절차만 따라 하게 된다

방법론을 산출물과 게이트에 넣는다

SDD · DDD · TDD · Deep Module

OPTIONAL

원칙은 흐름 안에서 작동한다



SDD·DDD · SDD·TDD · 사람 · TDD · Deep Module · DDD

SDD는 틀린 것을 빠르게 만들지 않는다

- 무엇을 만들지
- 무엇을 만들지 않을지
- 성공을 어떻게 확인할지

잘못된 방향으로 빨라진 것은 개선이 아니다

예전

- 잘못 이해한 요구
- 코드가 되기까지 며칠
- 중간에 알아챌 기회

AI와 함께

- 잘못 이해한 요구
- 30분 만에 완성
- 되돌릴 코드가 더 많음

OPTIONAL

긴 프롬프트와 명세는 다르다

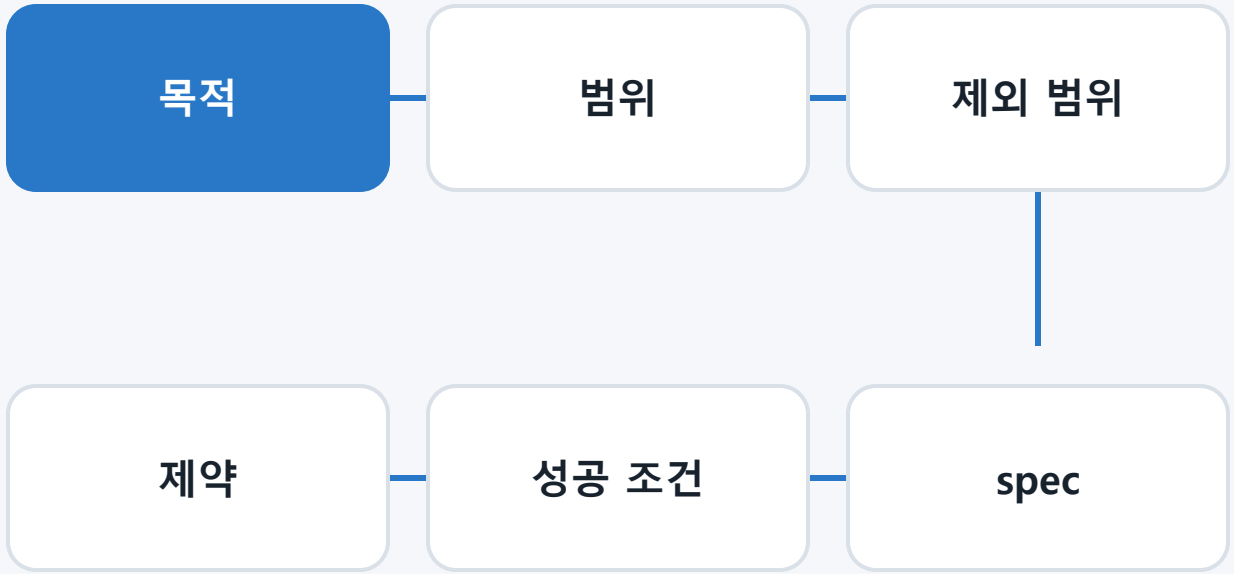
긴 프롬프트

- 답을 자세히 지시한다
- 숨은 모호함이 남을 수 있다
- 대화가 끝나면 사라진다

명세

- 질문으로 모호함을 찾는다
- 제외 범위를 합의한다
- 성공 조건이 파일로 남는다

합의가 파일로 남아야 다음 단계로 간다



대화가 아니라 남는 상태가 목적

DDD는 같은 단어를 같은 뜻으로 쓰는 데서 시작한다

- 모르는 용어를 추측하지 않는다
- 같은 정의를 사람·코드·테스트에 사용한다

이름이 흔들리면 설계와 테스트도 흔들린다

‘주문 취소’란 무엇인가?

- 주문 상태만 바꾸는가
- 재고를 복원하는가
- 결제 환불까지 포함하는가
- 배송 후에도 가능한가

용어집은 이름 판단을 대조 가능하게 만든다

용어집이 없을 때

- AI가 그럴듯한 이름을 만든다
- 세션마다 표현이 달라진다
- 사람만 어색함을 느낀다

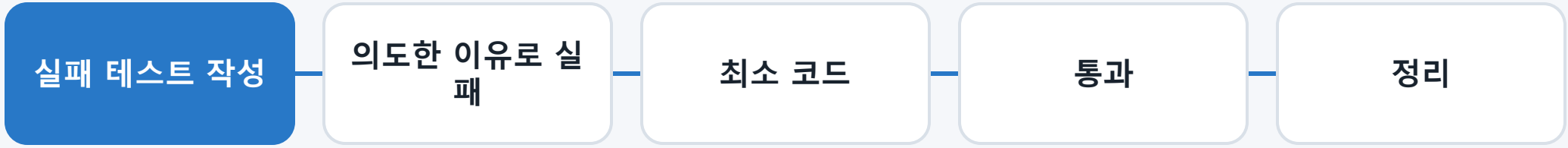
용어집이 있을 때

- 정의와 이름을 대조한다
- 모든 작업에 같은 맥락을 준다
- 애매하면 질문한다

TDD는 원하는 행동을 코드보다 먼저 정한다

- 테스트가 설계의 첫 문장이 된다
- 구현은 그 문장을 통과하는 최소 변경이다

빨강을 보지 않으면 테스트의 증거를 모른다



Red → Green → Refactor

OPTIONAL

TDD는 작은 검증 단위로 위임 범위를 넓힌다

- 요구 행동을 하나 고른다
- 실패 테스트를 실행한다
- 그 행동만 통과시키는 코드를 쓴다
- 관련 테스트를 다시 실행한다

DEEP MODULE

좋은 모듈은 내부가 아니라 외부를 단순하게 만든다

얕은 모듈

- 호출자가 내부 사정을 안다
- 옵션과 순서가 밖으로 샌다
- 변경이 여러 곳에 번진다

깊은 모듈

- 작은 인터페이스
- 큰 내부 복잡성을 숨긴다
- 불변조건과 실패 계약이 분명하다

OPTIONAL

Deep Module은 단계가 아니라 리뷰 질문으로 산다

- 호출자가 새로 알아야 할 것은 무엇인가
- 내부 복잡성이 밖으로 새었는가
- 인터페이스 없이 내부를 바꿀 수 있는가
- 실패 조건이 계약에 드러나는가

네 방법론은 서로 다른 질문을 남긴다

무엇을 합의하고 증명하나

- SDD — 무엇을 만들고 성공을 어떻게 아나
- DDD — 이 단어는 정확히 무엇인가
- TDD — 어떤 테스트로 증명하나

어떤 경계를 지키나

- Deep Module — 호출자는 무엇만 알면 되나
- 산출물 — spec·용어집·테스트·계약
- harness — 상태와 검증으로 강제

OPTIONAL

원칙은 문서가 아니라 다음 행동을 바꿔야 한다

01 합의가 없으면 구현하지 않는다

02 용어가 애매하면 추측하지 않는다

03 실패 테스트 없이 통과를 믿지 않는다

04 호출자에게 복잡성을 넘기지 않는다

합의 → 계획 → 구현 → 복원 → 지식

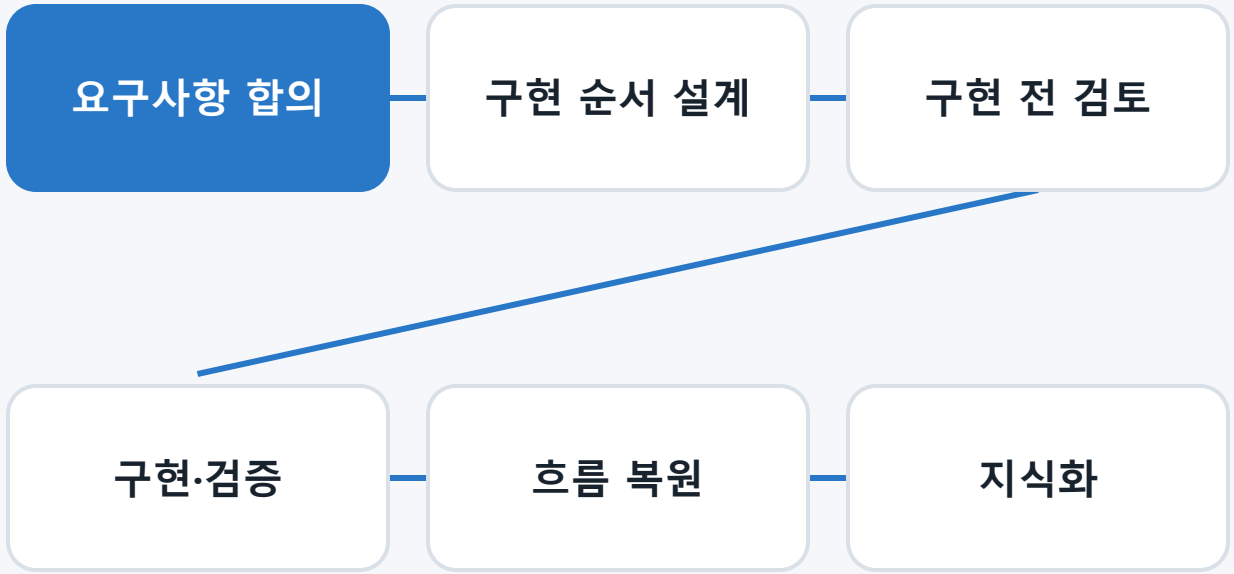
Harness의 여섯 단계

OPTIONAL

코드는 빨리 늘지만 이해는 자동으로 늘지 않는다

- 생성된 코드는 기술 부채뿐 아니라 인지 부채를 남긴다
- 다음 사람이 다시 조사하면 속도 이득이 사라진다

여섯 단계가 하나의 작업을 닫는다



spec · plan · explained · code + tests · e2e 문서 · wiki

OPTIONAL

안쪽 루프는 코드, 바깥 루프는 지식을 갱신한다

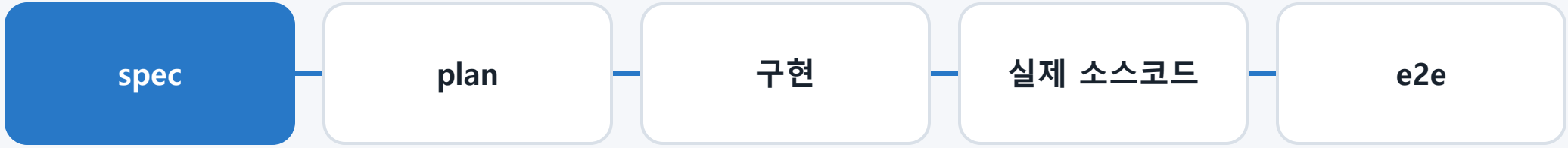
안쪽 루프

- 생성
- 검증
- 수정
- 재검증

바깥 루프

- 합의
- 구현
- 흐름 복원
- 다음 작업의 지식

정본은 구현 전후에 이동한다



무엇을 만들기로 했나 · 어떤 순서로 만들까 · 전환점 · 지금 뭐가 도나 · 코드에서 복원한 지도

OPTIONAL

단계를 건너뛰는 것은 실패가 아니라 판단이다

가볍게 처리

- 오타·상수 변경 — 기존 검증
- 원인 명확한 버그 — TDD 루프
- 기존 기능 확장 — plan부터

전체 흐름

- 새 기능
- 마이그레이션
- 구조 변경
- 원인 불명확한 버그

세 질문이 필요한 단계를 결정한다

- 다른 해석이 가능한가 → 합의
- 한 번에 증명하기 어려운가 → 계획
- 다음 사람이 다시 조사하는가 → 흐름 복원과 지식화

무엇을 만들지, 무엇을 만들지 않을지 정한다

목적 · 범위 · 제외 범위 · 제약 · 관찰 가능한 성공 조건

OPTIONAL

‘주문 취소’의 빈칸부터 찾는다

질문에서 드러난 것

- 환불을 포함하는가
- 배송 시작 후에도 가능한가
- 부분 취소가 있는가
- 누가 취소할 수 있는가

합의된 spec이 구현 전 정본이다

01 배송 전에만 취소 가능

02 부분 취소는 범위 밖

03 결제 취소는 별도 이벤트

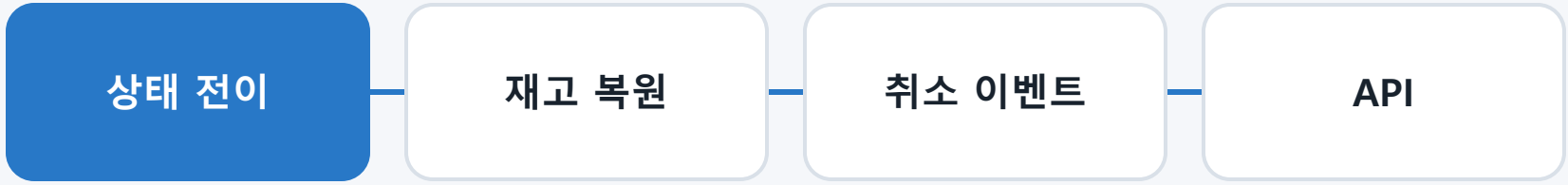
04 성공: CANCELLED + 재고 복원

Plan은 검증 가능한 순서를 정하는 실행 계약이다

- 어떤 행동을 먼저 증명할까
- 어느 파일과 테스트를 거칠까
- 무엇이 선행되어야 할까

OPTIONAL

가장 큰 위험을 먼저 증명한다



배송 시작 후 취소 금지 · 수량 복원 · 결제 취소와 분리 · 마지막에 연결

구현 전에 오해를 잡는 것이 가장 싸다

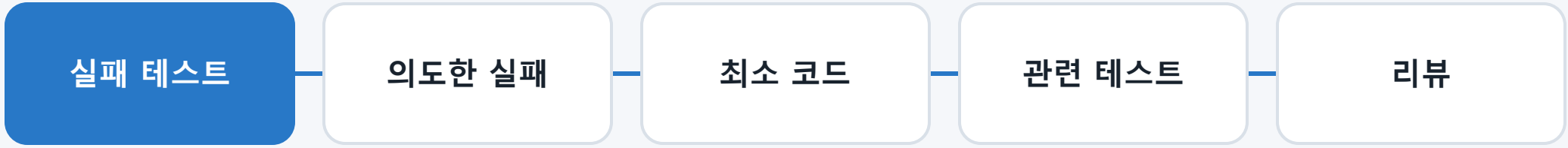
- 왜 이 설계인가
- 어떤 대안을 왜 기각했나
- 왜 이 task 순서인가

OPTIONAL

결제 취소를 같은 트랜잭션에 넣지 않은 이유

- 외부 PG 호출은 실패할 수 있다
- 주문 취소 자체는 완료되어야 한다
- 트랜잭션 커밋 후 이벤트로 넘긴다

실패하는 테스트와 최소 코드로 task를 닫는다



생성 → 검증 → 수정 → 재검증이 실제로 도는 자리

OPTIONAL

‘배송이 시작된 주문은 취소할 수 없다’를 먼저 증명한다

- 테스트를 작성한다
- 빨갛게 실패하는지 확인한다
- 상태 전이 가드를 추가한다
- 다시 실행해 통과시킨다

구현이 끝나면 정본의 질문이 달라진다

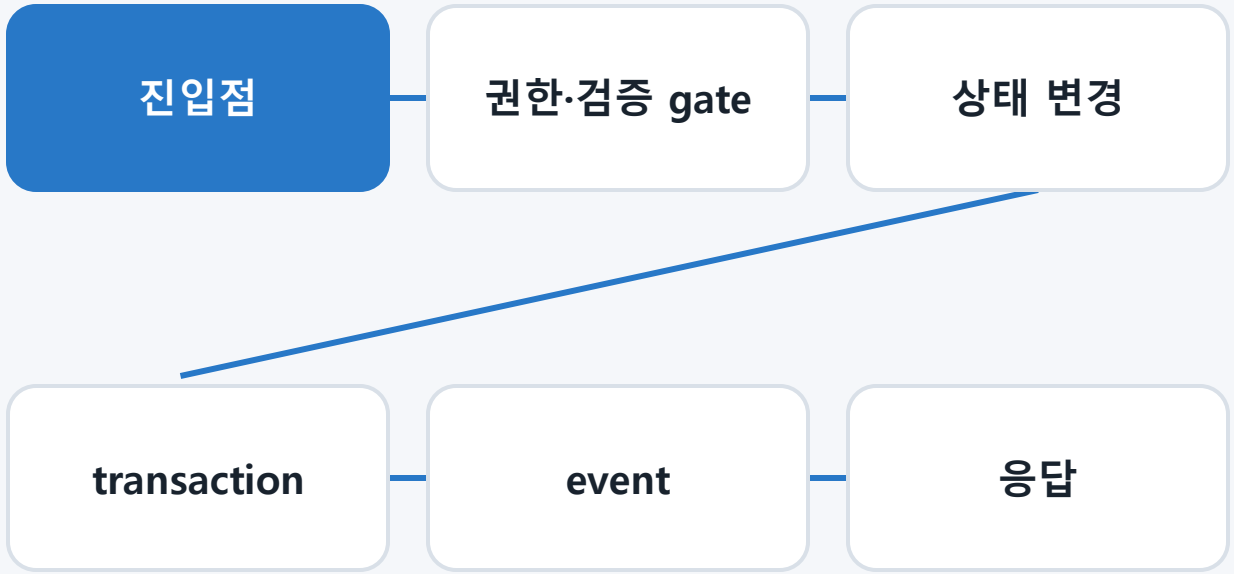
구현 전

- spec — 무엇을 만들기로 했나
- plan — 어떤 순서로 만들까
- explained — 왜 이 설계인가

구현 후

- 실제 소스코드 — 지금 뭐가 도나
- 테스트 — 어떤 행동이 증명됐나
- plan — 관련 파일을 찾는 지도

실제 코드에서 front → back → db → back → front를 되읽는다



plan과 다른 지점을 숨기지 않고 보고한다

OPTIONAL

문서와 코드의 차이는 판단을 요구하는 발견이다

발견된 차이

- 의도된 개선
- 요구에서 벗어남
- spec이 애매했음

돌아갈 곳

- spec을 갱신
- 구현을 되돌림
- brainstorming으로 돌아감

OPTIONAL

끝난 작업이 다음 작업의 입력이 된다

01 spec — 합의

02 plan — 구현 지도

03 explained — 이유

04 e2e — 실제 흐름

05 wiki — 원문을 찾는 색인

시간이 줄어들면 모듈째 덜어낸다

5시간

- 전체 70장
- AI 기초와 방법론 사례 포함
- 여섯 단계 상세 사례

3시간 / 2시간

- 3시간 — 선택 모듈 생략
- 2시간 — 원문·다섯 축·검증 루프·정본 이동 유지
- 개별 장이 아니라 연속 묶음으로 생략

검증할 수 있는 만큼만 맡길 수 있다

01 AI가 일하는 환경을 설계한다

02 검증 결과를 자동으로 되먹인다

03 판단과 책임은 사람에게 남긴다

04 나는 이 코드에 대해 책임질 수 있는가?